



SUNBEAM
Exploring New Ideas Reaching New Heights



A
PROJECT REPORT
ON
Infotainment System

Carried Out at



CENTRE FOR DEVELOPMENT OF ADVANCED COMPUTING
SUNBEAM INFORMATION & TECHNOLOGY PUNE

Submitted By
Raghav Goel (250844230069)
Aishwarya Chavan (250844230006)
Pranali Jamdade (250844230066)
Trupti Gadag (250844230092)

PG DIPLOMA IN EMBEDDED SYSTEMS & DESIGN
C-DAC, Sunbeam Pune



INDEX

Sr No.	Contents	Page No.
1.	Introduction	3
2.	Abstraction	4
3.	Block Diagram	5
4.	Required Component	6-9
5.	CAN Communication	10
6.	Interfacing Diagrams	11-12
7.	Device drivers	13
8.	Future Scope	14
9.	Results	15-16
10.	Conclusion	17



INTRODUCTION

Modern vehicles increasingly rely on intelligent infotainment systems to enhance driver safety, comfort, and connectivity. This project presents an **RTOS-based automotive infotainment system** built using the **STM32F411 microcontroller** as the primary controller and **BeagleBone Black** as the display and processing unit.

The system integrates multiple modules including an **APDS sensor** for proximity, gesture, and ambient light sensing, and a **GPS module** for real-time location tracking using NMEA GGA and RMC sentences to extract latitude, longitude, date, and time. These inputs are managed through dedicated RTOS tasks, ensuring deterministic and real-time operation.

Processed data from STM32 is transmitted over **CAN communication** using an **MCP2515 SPI CAN controller** with **TJA1050 transceiver**. The BeagleBone Black receives this data through its SPI driver and displays the information on an LCD using a custom I2C device driver.

Gesture control enables touchless interaction, while GPS provides tracking capability. The APDS ambient light sensor can also be used to automatically adjust LCD brightness. The system architecture demonstrates a practical automotive embedded design combining sensors, RTOS, communication protocols, and Linux-based processing.

This project simulates a real automotive infotainment platform capable of expansion toward GSM calling, vehicle parameter monitoring (fuel level, speed), and map-based navigation.



ABSTRACTION

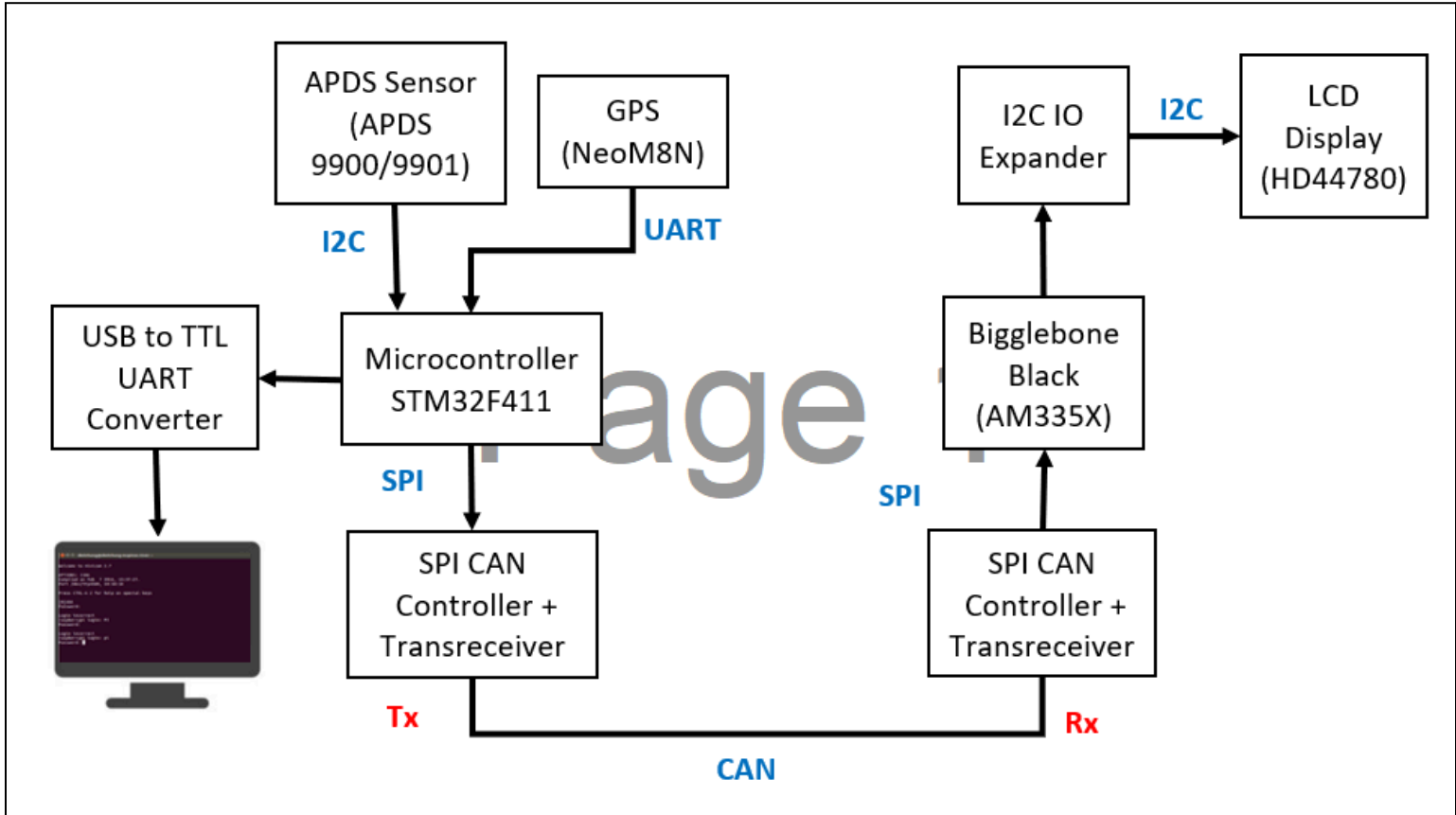
This project implements a **real-time automotive infotainment system** using STM32F411 and BeagleBone Black interconnected via CAN bus. The STM32 operates under RTOS, managing multiple concurrent tasks such as gesture sensing, proximity detection, ambient light monitoring, and GPS data acquisition.

GPS data is parsed from GGA and RMC NMEA sentences to obtain location and timing information. The APDS sensor provides gesture and proximity inputs for user interaction, along with ambient light sensing for adaptive display brightness.

Sensor and GPS data are transmitted over CAN using MCP2515 and TJA1050. The BeagleBone Black receives this data through SPI, processes it in user space, and displays results on an LCD via an I2C device driver.

The system demonstrates modular design, real-time scheduling, and reliable inter-processor communication, making it suitable for automotive infotainment and vehicle monitoring applications. Future extensions include GSM calling, live vehicle parameters over CAN, cloud connectivity, and graphical navigation.

BLOCK DIAGRAM



COMPONENTS REQUIRED

● Hardware Components

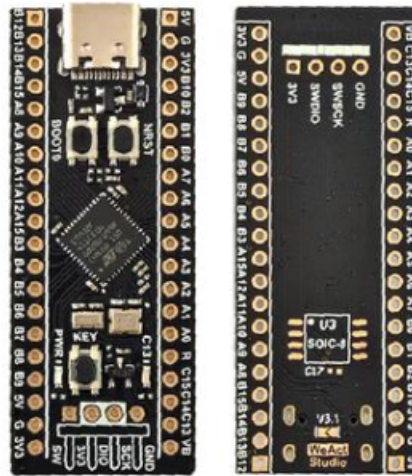
Sr.	COMPONENT NAME	QUANTITY
1.	STM32F411 Microcontroller	1
2.	BeagleBone Black	1
3.	APDS Sensor (Proximity, Gesture, Ambient Light)	1
4.	GPS Module	1
5.	SPI CAN Controller & Transceiver Module	2
6.	LCD Display	1
7.	Connecting wires	As per requirement

● Software Components

- RTOS (FreeRTOS) on STM32F411
- SPI Driver (CAN communication)
- I2C Device Driver (LCD interface)
- Linux OS on BeagleBone Black

1) STM32F411 Microcontroller

The STM32F411 is a high-performance 32-bit microcontroller based on the ARM Cortex-M4 core with floating point unit. It operates up to 100 MHz and includes up to 512 KB Flash memory and 128 KB SRAM. The device provides multiple communication interfaces including SPI, I2C, USART, and CAN (via external controller), along with timers, ADC, DMA, and GPIO peripherals. It supports low-power operating modes and is designed for embedded applications requiring real-time performance.



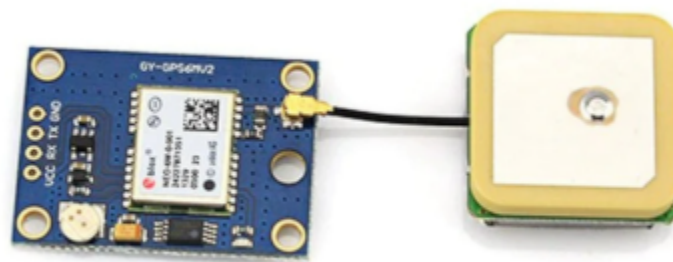
**STM32F411CEU6 BlackPill
Development Board**

2) APDS Sensor (APDS 9900 – Proximity, Ambient Light)

The APDS sensor integrates proximity sensing, gesture recognition, and ambient light sensing in a single module. It uses infrared LEDs and photodiodes for proximity and gesture detection, and a photodiode array for ambient light measurement. The device communicates via I2C interface and supports programmable gain, integration time, and interrupt functionality. It is intended for touchless user interfaces and automatic brightness control applications.

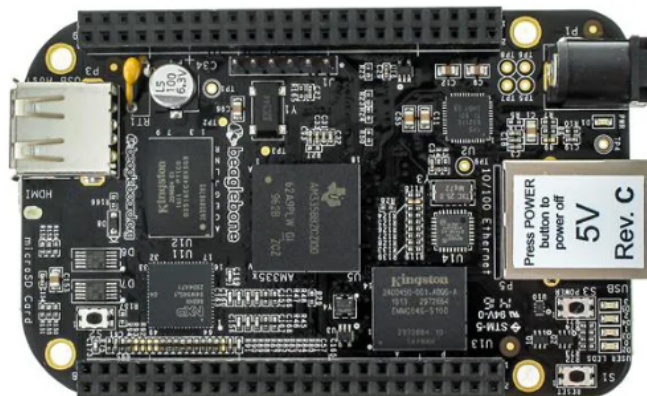
3) GPS Module (NEO-M8N)

The GPS receiver module provides positioning data by receiving signals from multiple GPS satellites. It outputs standard NMEA sentences such as GGA and RMC over UART, which include latitude, longitude, altitude, date, time, and fix status. The module supports cold, warm, and hot start operation and typically operates at 3.3 V or 5 V supply depending on the model.



4) BeagleBone Black (AM335x)

The BeagleBone Black is a single-board computer based on the TI AM335x ARM Cortex-A8 processor. It supports Linux operating systems and provides interfaces including SPI, I2C, UART, GPIO, USB, Ethernet, and HDMI. It is commonly used for embedded Linux development and supports external peripherals through expansion headers.



5) HD44780 Compatible LCD

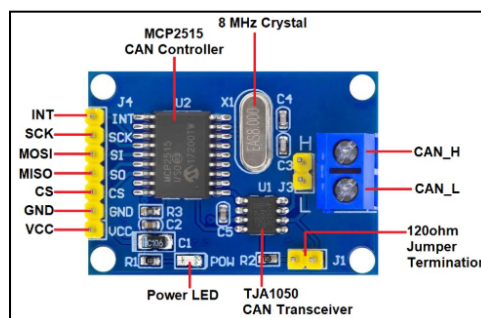
The HD44780 LCD controller drives character LCD modules such as 16×2 displays. It supports 4-bit or 8-bit parallel data interface and provides internal DDRAM for character storage. The controller supports one-line or two-line display modes, cursor control, and custom character generation via CGRAM.

6) SPI CAN Controller & Transceiver Module (MCP2515 + TJA1050)

The MCP2515 + TJA1050 CAN module integrates the Microchip MCP2515 standalone CAN controller and the NXP TJA1050 high-speed CAN transceiver on a single board. The MCP2515 communicates with the host microcontroller through an SPI interface and implements the CAN 2.0B protocol, supporting both standard and extended frames. It provides internal transmit and receive buffers, acceptance filters, masks, and interrupt capability.

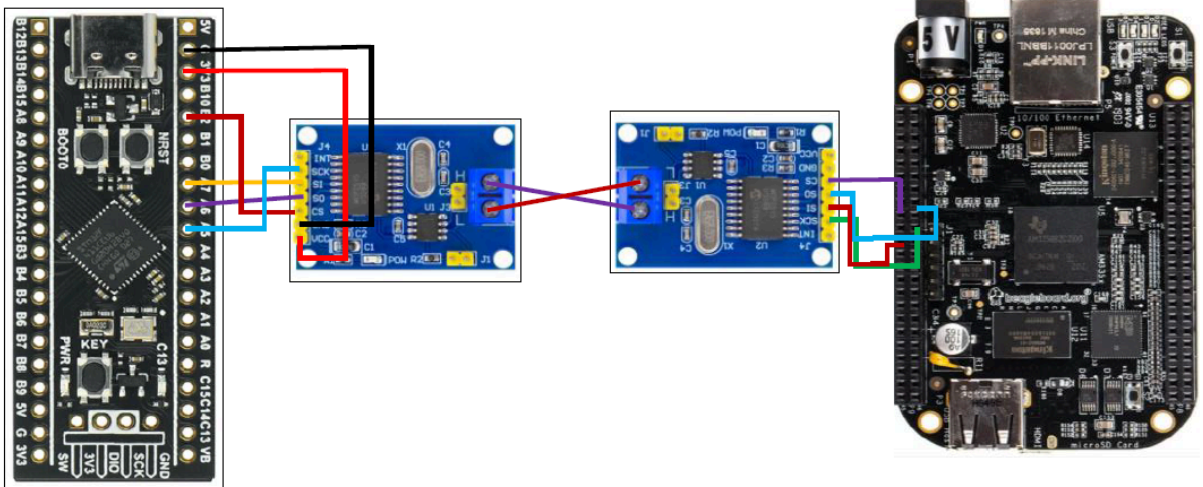
The TJA1050 acts as the physical layer interface between the MCP2515 and the CAN bus. It converts logic-level CAN signals from the controller into differential CANH and CANL signals compliant with ISO11898 and supports data rates up to 1 Mbps. The transceiver includes protection features such as thermal shutdown and short-circuit protection.

This combined module simplifies CAN implementation by providing both protocol handling and physical layer communication, making it suitable for automotive and industrial networking applications.

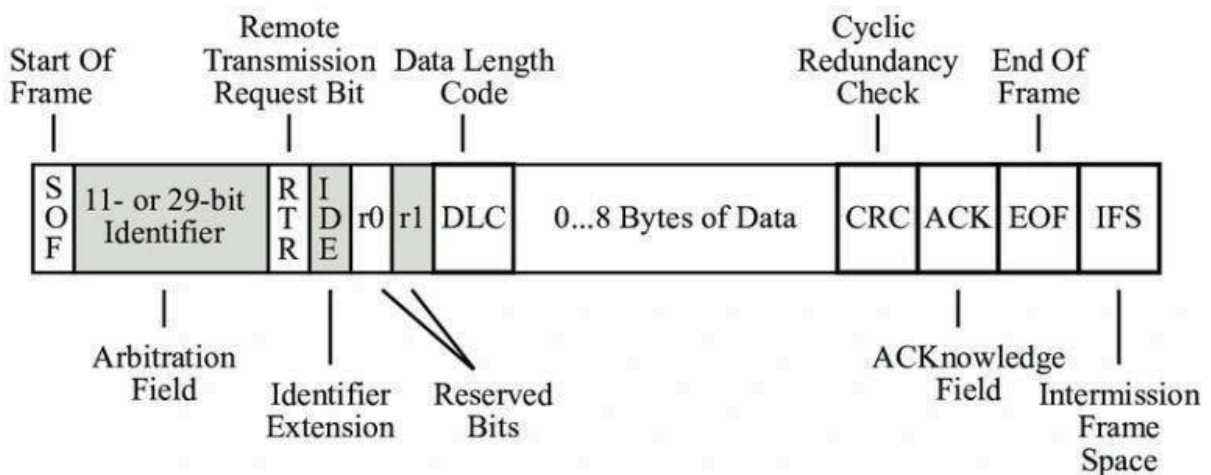


CAN COMMUNICATION

- CAN Communication Flow Between STM32F411 and BeagleBone Black (BBB)



- CAN Frame:



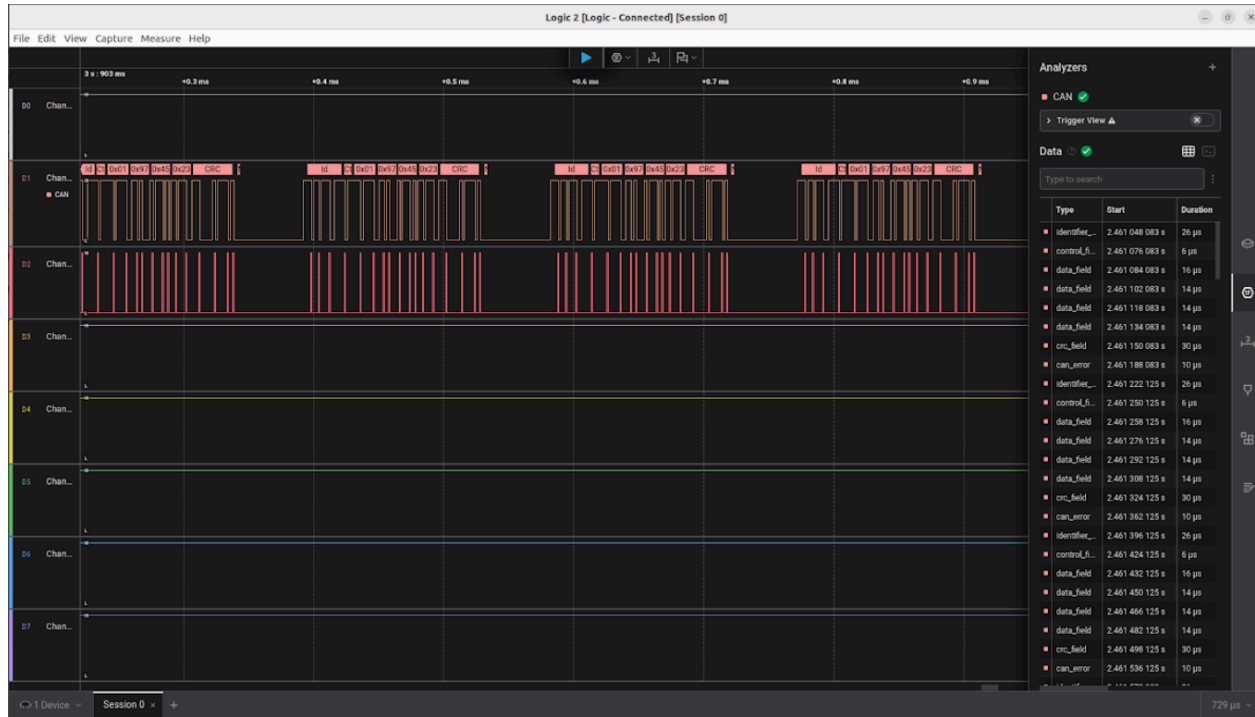


Fig. CAN result on Logic analyzer

APDS & GPS Interfacing with STM32F411

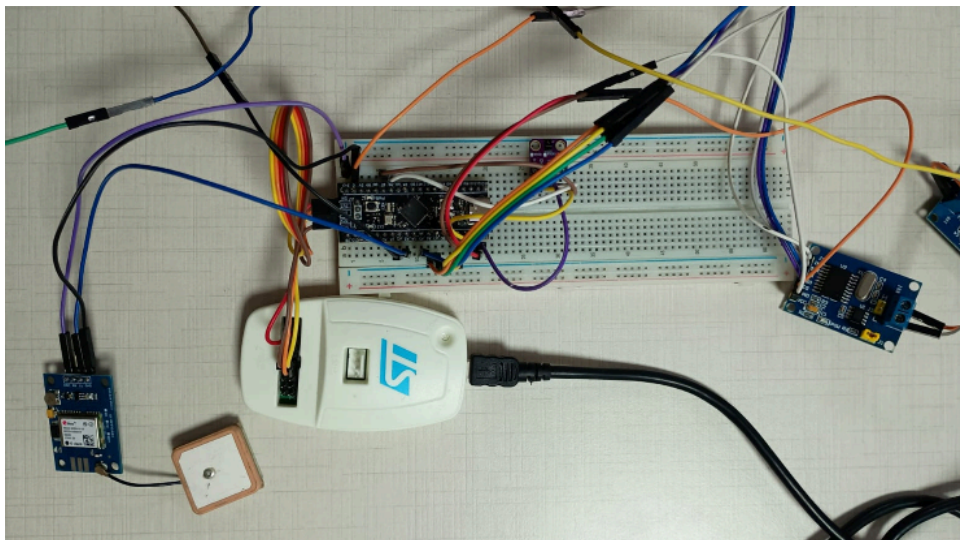


Fig1. APDS & GPS Interfacing with STM32F411

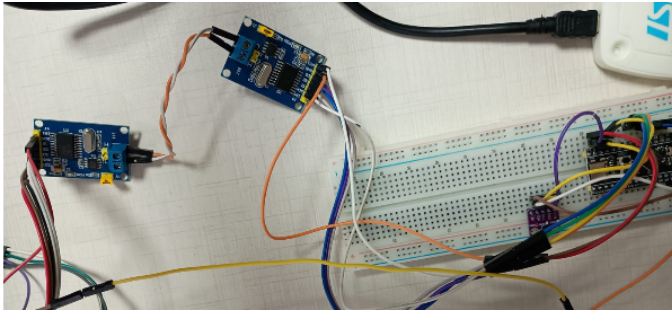


Fig2. CAN Communication

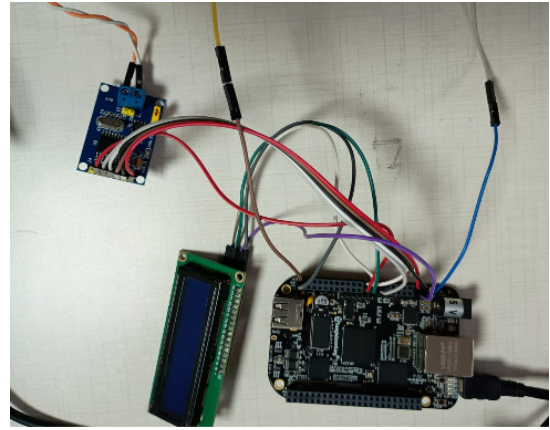
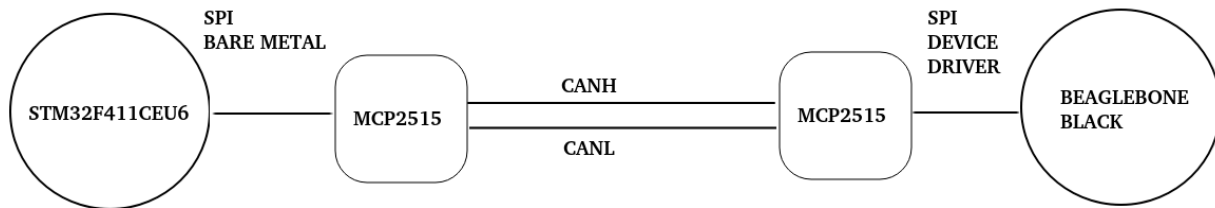


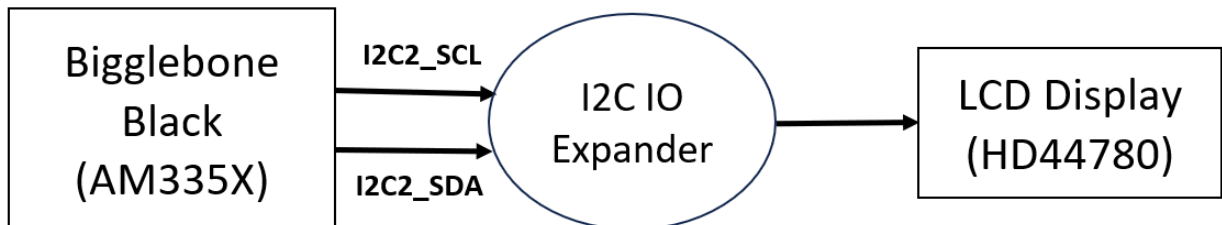
Fig3. I2C Driver Connection

DEVICE DRIVERS

SPI Device Driver Flow:



LCD (I2C) Device Driver:





FUTURE SCOPE

1) Automatic Display Brightness Control using APDS ALS:

Utilize the Ambient Light Sensor (ALS) of the APDS to automatically adjust LCD brightness based on surrounding light conditions.

In bright daylight, the system increases display brightness for better visibility. In low-light or night conditions, brightness is reduced to avoid driver distraction and save power. This can be implemented as a dedicated RTOS task, where ALS values are periodically read and mapped to PWM duty cycle controlling the LCD backlight.

2) GSM Calling & Messaging Facility:

Integrate a GSM/LTE module to enable hands-free calling and SMS alerts. Incoming call information (caller ID, call status) can be displayed on the LCD, improving driver convenience and safety.

3) Real-Time Vehicle Parameter Display via CAN:

Extend CAN communication to fetch live vehicle data such as fuel level, speed, RPM, engine temperature, and battery status, and display multiple parameters simultaneously on the LCD as a digital dashboard.

4) Map-Based Navigation System:

Upgrade from raw GPS coordinates to full **map navigation**, including route guidance and turn-by-turn directions on the LCD using BeagleBone graphical capabilities.

5) IoT Cloud Data Upload & Remote Monitoring:

As a future scope, we can add IoT connectivity to push GPS and CAN vehicle data to the cloud for remote monitoring, fleet tracking, and analytics using GSM or Wi-Fi

RESULTS

Driver Loaded inside Kernel:

```

[ 670.918466] main : lcd Probed!!!
[ 670.918466] main : LCD is initialized.
[ 670.918466] main : alloc_chrdev_region() allocated device number 236/0.
[ 670.918466] main : class_create() created device class.
[ 670.918466] main : device_create() created device file.
[ 670.918466] main : cdev_add() device added into kernel.
[ 670.918466] main : lcd_open() is called
[ 670.918466] main : lcd_write() is called
[ 670.918466] main : kmalloc allocation successful.
[ 670.918466] main : lcd_close() is called
[ 670.918466] main : lcd_open() is called
[ 670.918466] main : lcd_write() is called
[ 670.918466] main : kmalloc allocation successful.
[ 670.918466] main : lcd_close() is called
[ 670.918466] main : lcd_open() is called
[ 670.918466] main : lcd_write() is called
[ 670.918466] main : kmalloc allocation successful.
[ 670.918466] main : lcd_close() is called
[ 670.918466] main : lcd_open() is called
[ 670.918466] main : lcd_write() is called
[ 670.918466] main : kmalloc allocation successful.
[ 670.918466] main : lcd_close() is called
[ 670.918466] main : cdev_del() deleted cdev from kernel.
[ 670.918466] main : device_destroy() destroyed device file.
[ 670.918466] main : class_destroy() destroyed device class.
[ 670.918466] main : unregister_chrdev_region() deallocated the device number.
[ 670.918466] main : lcd Removed!!!
[ 1050.599520] main : lcd Probed!!!
[ 1050.599520] main : LcdInit!!!
[ 1050.599520] main : LCD is initialized.
[ 1050.599520] main : alloc_chrdev_region() allocated device number 236/0.
[ 1050.599520] main : class_create() created device class.
[ 1050.599520] main : device_create() created device file.
[ 1050.599520] main : cdev_add() device added into kernel.
[ 1050.599520] main : desd_lcd_driver is added
[ 1050.599520] main : lcd_open() is called
[ 1050.599520] main : lcd_write() is called
[ 1050.599520] main : kmalloc allocation successful.
[ 1050.599520] main : lcd_close() is called
[ 1583.424532] My_SPI_Driver: SPI_DRIVER_INIT() is called
[ 1583.424532] My_SPI_Driver: SPI_LCD_PROBE() is called
[ 1583.424532] My_SPI_Driver: OK tx_buf=0xC0 rx_buf=0x00

```


SPI Driver output:

```
[ 4990.135399] My_SPI_Driver: OK tx_buf=0x67 rx_buf=0x00
[ 4990.135407] My_SPI_Driver: OK tx_buf=0x00 rx_buf=0xD0
[ 4990.135513] My_SPI_Driver: OK tx_buf=0x03 rx_buf=0x00
[ 4990.135523] My_SPI_Driver: OK tx_buf=0x68 rx_buf=0x00
[ 4990.135530] My_SPI_Driver: OK tx_buf=0x00 rx_buf=0x01
[ 4990.135633] My_SPI_Driver: OK tx_buf=0x03 rx_buf=0x00
[ 4990.135642] My_SPI_Driver: OK tx_buf=0x69 rx_buf=0x00
[ 4990.135650] My_SPI_Driver: OK tx_buf=0x00 rx_buf=0x6E
[ 4990.135753] My_SPI_Driver: OK tx_buf=0x02 rx_buf=0x00
[ 4990.135763] My_SPI_Driver: OK tx_buf=0x2C rx_buf=0x00
[ 4990.135771] My_SPI_Driver: OK tx_buf=0x00 rx_buf=0x00
[ 4990.135780] My_SPI_Driver: data: 01 d0 01 6e
[ 4990.135797] My_SPI_Driver: kfifo_to_user() is successful
[ 4990.136655] main : lcd_write() is called
[ 4990.136674] main : kmalloc allocation successful.
[ 4991.647537] main : lcd_close() is called
[ 4991.647579] My_SPI_Driver: my_spi_close() is successful
[ 5295.229046] My_SPI_Driver: SPI_DRIVER_EXIT() is called
[ 5295.229137] My_SPI_Driver: SPI_LCD_REMOVE() is called
[ 5295.237875] My_SPI_Driver: SPI_DRIVER_EXIT() is completed
debian@BeagleBone:~/userBuffer$
```

Application program output:

```
[ 1583.447835] My_SPI_Driver: SPI_DRIVER_EXIT() is completed
debian@BeagleBone:~/userBuffer$ sudo ./a.out
spi device file opened.
lcd device file opened.
bytes read from device: 8 -- 11 32 44 33
bytes read from device: 8 -- 00 00 99 29
bytes read from device: 8 -- 00 e0 ff ff
bytes read from device: 8 -- 00 e0 ff ff
bytes read from device: 8 -- 00 e0 ff ff
bytes read from device: 8 -- 00 e0 ff ff
bytes read from device: 8 -- 00 e0 ff ff
bytes read from device: 8 -- 00 e0 ff ff
bytes read from device: 8 -- 00 e0 ff ff
bytes read from device: 8 -- 11 32 44 33
bytes read from device: 8 -- 00 e0 ff ff
bytes read from device: 8 -- 41 92 04 13
bytes read from device: 8 -- 00 e0 ff ff
bytes read from device: 8 -- 00 00 99 29
bytes read from device: 8 -- 00 e0 ff ff
bytes read from device: 8 -- 00 e0 ff ff
bytes read from device: 8 -- 00 e0 ff ff
bytes read from device: 8 -- 00 e0 ff ff
bytes read from device: 8 -- 00 e0 ff ff
bytes read from device: 8 -- 00 e0 ff ff
```




CONCLUSION

The implemented infotainment system successfully demonstrates an **RTOS-based multi-sensor embedded architecture** with reliable CAN communication between STM32 and BeagleBone Black. Gesture and proximity sensing enable intuitive interaction, while GPS provides accurate location and timing data. The use of MCP2515 and TJA1050 ensures robust automotive-style communication, and custom SPI and I2C drivers validate low-level driver development skills.

The project highlights practical integration of **RTOS task scheduling, sensor interfacing, GPS parsing, CAN networking, and Linux device drivers**, closely reflecting real automotive embedded systems. Additionally, ambient light-based brightness control improves usability and power efficiency.

Overall, this project forms a strong foundation for advanced infotainment platforms and can be extended to include GSM calling, live vehicle diagnostics, map navigation, cloud connectivity, and ADAS features—making it highly relevant for modern automotive applications.